

Mini Project: Build an Alert Triage Agent with LangGraph

Build a small agent that decides which alerts to escalate, measure both ways it can be wrong, and say what a graph gives you that a chain does not.

Runs on your own machine or Colab • 100 points

1. What this project is about

A chain runs straight through. An agent decides. That decision is what you study here. You build a small triage agent with LangGraph: a state, a few nodes, a conditional edge, a tool, and memory. Then you give it labelled security alerts and count the two ways it can be wrong, the incident it missed and the routine alert it escalated anyway. Widening the rule trades one against the other, and choosing where to sit on that trade off is the work of the project.

Part 1 is practice: work through the two lecture notebooks in this module, `langgraph_state_nodes` and `langgraph_branch_tools`, then finish five short pieces of code in the starter notebook. The rest is the project proper. You write your own labelled alerts, run the agent on them, change the graph itself rather than only the word list, and report what you found.

Everything runs offline. Every node is ordinary Python, so there is no API key, no model download, and no GPU. That is deliberate: a deterministic agent means the numbers you report are exact and reproducible.

2. Using AI assistants

You may use ChatGPT, Claude, Copilot, or any other assistant to write code, fix errors, and organize results. What is assessed is whether you understand what you built. In the video you will walk through your own notebook and explain how the state moves through the graph and why it routed the way it did. Anything you cannot explain will not earn credit, however well it runs.

3. Part 1: practice building the graph

10 points

Open `MiniProject_Agent_Starter.ipynb`. Parts 1 to 5 contain five short pieces of code, each marked with a `TODO` banner. Work from the two lecture notebooks: `langgraph_state_nodes` covers state and nodes, and `langgraph_branch_tools` covers routing, tools, and memory.

	You write	Lines
TODO 1	the state, a typed dictionary of what an alert carries	2
TODO 2	a node, a function returning only the fields it changed	2
TODO 3	the router, which returns the label of the next branch	1
TODO 4	a node that calls a tool	2
TODO 5	build and compile: entry point, conditional edge, compile	3

Each cell states the output to expect. With the keyword rule from the lecture the agent misses half the real incidents on the provided alerts, which is the starting point rather than a bug.

4. Part 2: your own alerts and your own graph

30 points

From Part 9 of the notebook onward, no code is provided.

Copy `alerts_template.json` to `my_alerts.json` and write 12 labelled alerts, 6 you would escalate and 6 you would close. Base them on real alert types such as Windows event logs, EDR detections, cloud audit trails, or IDS signatures, write them in your own words, and record any sources in `SOURCES.md`. Include at least two hard cases: a real incident whose text contains none of your keywords, and a routine alert that contains one. For every alert you must be able to say why an analyst would escalate or close it. You are labelling, not guessing.

Then run the agent on your alerts, and make at least one change to the graph itself rather than to the word list. Add a node, add a tool, change the router, or add a second decision. A tool here is an ordinary Python function that a node calls, like the lookup already in the starter notebook, so an asset lookup or a reputation stub you write yourself is exactly what is meant. Measure the same alerts again and be ready to say what the change bought you.

5. Part 3: measure the two errors

15 points

Score three outcomes on your own alerts: caught, missed, and over escalated.

	Meaning	What it costs
caught	escalated something that deserved it	what you want
missed	closed a real incident	an attacker continues, unnoticed
over escalated	escalated routine noise	analyst hours, and eventually alert fatigue

Change one thing at a time, record what each change costs on the two error columns, and save the table as `results_mine.csv`. Never report a single accuracy figure: it hides which of the two errors you made, and which one you made is the whole question.

6. What to hand in

Submit this

one PDF report. On its first page, put the link to your project folder on GitHub and the link to your video. Everything else lives in the repository.

6.1 Your repository (5 points)

Use the private repository from the earlier projects, with your instructor and the class TA as collaborators, and add a new folder for this one:

```
your-repo/
  langgraph_project/
    README.md          what the project is, in a short paragraph
    agent_notebook.ipynb your completed notebook, with all outputs saved
    my_alerts.json      your 12 labelled alerts
    SOURCES.md          any sources you used
    results_mine.csv    your results
```

Run the notebook top to bottom and save it with its outputs showing before you commit.

6.2 Report (25 points)

A PDF. Begin with the concept check, five short answers of two or three sentences each, in your own words:

- What is the state in a LangGraph agent, and how do nodes use it?
- What is a node and what is an edge? What does the end marker do?
- How does a conditional edge let an agent choose its path? Give a security example.
- What is a tool in an agent, and why do agents need them?
- What does a checkpoint with a thread identifier give the agent?

Then the analysis:

1. What you built, briefly: the graph in a sentence or a sketch, your 12 alerts, and why you labelled the two hard cases the way you did.
2. The two errors. Which would you rather your agent made, and why? Answer as the person who has to staff the team, not in the abstract.
3. Your operating point. Which rule would you deploy, and which alerts are you choosing to get wrong?
4. The alerts that stay wrong at every setting. Explain what a keyword rule cannot see, describe the change you made to the graph, and say what it bought you.
5. LangGraph compared with LangChain. What did the graph give you that a chain could not? Point at your conditional edge and at memory by thread identifier, using your own output as evidence, and say when a chain would still be the better choice.
6. Honest limits. Twelve alerts means one alert is a sixth of a column. How much confidence does that justify, and what would you measure next?

6.3 Video (15 points)

Five to ten minutes of screen recording with your narration. Walk through your notebook while it runs, trace one alert through the graph and explain each decision, show a caught alert and a missed one, and talk through your results. Live screens, not slides of screenshots.

7. Grading

Part	Points
Part 1, practice graph: five TODOs complete and the notebook running	10
Part 2, your 12 labelled alerts and your own change to the graph	30
Part 3, the two errors measured, one change at a time, <code>results_mine.csv</code>	15
Repository: private, instructor and TA added, separate folder, outputs saved	5
Report, including the concept check and the comparison with LangChain	25
Video	15
Total	100

Write your alerts yourself. Do not paste real alerts from an employer console; describe the type of alert instead. No API key or network access is needed anywhere in this project. If you choose to call a real service from a tool, it must be read only and pointed at lab or authorized systems.